

Reviewer Pack — Minimal Order of Diophantine Approximation and Resolution Pr...

Entry 2e92aac9c820 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 14:55:16 UTC

Minimal Order of Diophantine Approximation and Resolution Proof Width Inequality

Entry ID: 2e92aac9c820 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 14:55:16 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Approximation) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

For every CNF formula φ , the minimal order of Diophantine approximation of its Boolean function $f(\varphi)$ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\log(\text{mo}(f)) = \Theta(w(\varphi))$ for some constant mo .

Rationale (proposer's reasoning):

Diophantine approximation measures the distance of a function to the nearest algebraic number, which could reflect the structure of the Boolean function. Resolution proofs are a symbolic method for proving propositional logic formulas and have been shown to be related to circuit complexity. The conjecture suggests that this geometric property of functions might offer insights into the complexity of resolution proofs.

Taxonomy category: resolution_proofs (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4f6dc6a1bfb0fe0d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of the correlated pairs $(\log(\text{mo}(f)), w(\varphi))$ fall within a range where their ratio is between 0.5 and 2 for all seeds, and no seed produces a pair with a ratio less than 0.5 or greater than 2.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Diophantine Approximation' AND 'resolution proof complexity' - 'minimal order Diophantine approximation' AND 'proof width inequality' AND 'CNF formula' - 'Boolean function' AND 'order of Diophantine approximation' AND 'linear correlation' AND 'resolution proof width'

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.randint(-1, 0) * (i + 1) for i in range(n)]
            if all(x == 0 for x in clause):
                continue
            clauses.append(clause)
        return clauses

    def boolean_function(cnf, n):
        assignment = {i: random.choice([True, False]) for i in range(1, n + 1)}
        return any(all(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in cnf)

    def diophantine_approximation(f, n):
        x = 1.5
        f_n = f(n)
        approx = round(x * f_n)
        return abs(approx - f_n) / f_n

    def resolution_width(cnf):
        width = 0
        for clause in cnf:
            if len(clause) > width:
                width = len(clause)
        return width
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f_n = boolean_function(generate_cnf(n), n)
    mo_f = diophantine_approximation(f_n, n)
    w_n = resolution_width(generate_cnf(n))

    if mo_f == 0 or w_n == 0:
        continue

    ratio = math.log(mo_f) / w_n
    results.append((ratio, f_n, w_n))

if not results:
    return {
        "metric_name": "log_mo_over_w",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "no_valid_pairs"
    }

ratios = [r for r, _, _ in results]
mean_ratio = sum(ratios) / len(ratios)
std_ratio = math.sqrt(sum((r - mean_ratio)**2 for r in ratios) / len(ratios))

conjecture_holds = all(0.5 <= r <= 2 for r, _, _ in results)
counterexample = "" if conjecture_holds else "out_of_bounds"

return {
    "metric_name": "log_mo_over_w",
    "metric_value": mean_ratio,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(r["conjecture_holds"] for r in results):
        support_fraction = 1.0
    else:
        support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)} st
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"out_of_bounds\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1708cac8.py", line 93, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1708cac8.py", line 51, in run_trial
    f_n = boolean_function(generate_cnf(n), n)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1708cac8.py", line 32, in boolean_function
    return any(all(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in cnf)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1708cac8.py", line 32, in <genexpr>
    return any(all(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in cnf)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1708cac8.py", line 32, in <genexpr>
    return any(all(assignment[abs(lit)] == (lit > 0) for lit in clause) for clause in cnf)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

KeyError: 0

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means it was unable to verify the conjecture's support conditions. | next: Re-run the test with appropriate error handling and logging to ensure that it can complete without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13614
2	propose	ollama_remote	glm4:latest	0	0	12509
3	preregistration	ollama_remote	glm4:latest	0	0	9516
4	novelty	ollama_remote	glm4:latest	0	0	8596
5	novelty	ollama_remote	glm4:latest	0	0	9237
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15633
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10217
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12353
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10419
10	judge	ollama_remote	glm4:latest	0	0	17230

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119324 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/2e92aac9c820.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/2e92aac9c820.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/2e92aac9c820.tar.gz* (if generated)